使用嫁接法并行加速 baseml

Lintao Luo [1*]

[1] 重庆医科大学基础医学院，重庆市渝中区石油路，400016，中国

联系方式:
giantlinlinlin@gmail.com

[*] 第一作者

现地址: 重庆市渝中区石油路，400016，中国

数据可用性: 数据和可用代码详见GitHub

摘要

本文档介绍一个基于嫁接法的 baseml 并行加速方案，旨在提升 PAML 软件进行大规模样本最大似然推断的效率。

方法学实现逻辑 Methods

中文

总体框架

本研究实现了一套面向大规模系统发育树枝长校正的分治式计算流程，其核心思想是将全树问题拆解为 “全局骨架约束 + 局部目标子树估计 + 全局回拼接 + 超度量标准化” 四个连续阶段。该流程始终以同一棵主树拓扑为参照，在局部分析阶段反复保留共享骨架和统一外群，以便在局部最大似然估计结束后，将不同子树中的枝长信息重新投射回同一全局尺度。整个方法并不试图在局部分析中重新定义全树拓扑，而是将拓扑保持视为硬约束，将枝长校正视为主要优化对象。

该流程的输入包括一棵大规模系统发育树、与树端点相匹配的多序列比对、外群信息以及最大似然模型参数模板。输出则依次包括标准化后的主树、骨架树、目标子树、PAML 分析子树、局部枝长估计结果、全局合并后的非超度量树以及最终的超度量树。为保证可重复性，流程在每一阶段都显式保存中间结果、清单表和验证报告，使得每一步的结构定义、样本组成和枝长更新都可以被独立审计。

主树预处理与定根标准化

流程首先对输入树进行定根状态识别。若输入树尚未定根，则依据用户指定的外群集合进行严格外群定根；若输入树已经定根，则保留原始定根状态。随后，对中间主树进行结构标准化，以确保其满足 “根部含单一外群子分支” 的约束，并尽可能转化为二叉树形式。在线性链式的单子节点结构存在时，流程会折叠冗余节点；当出现多分叉时，则通过引入零长度的中间节点将其规整为二叉拓扑。该步骤的目的不是改变系统发育关系，而是为后续的切树、合树和拓扑一致性校验提供统一且可计算的结构基础。

在完成定根和二叉化之后，方法会为每个内部节点分配稳定标识，并记录节点到根的距离、树端点顺序以及输入文件指纹信息。这些预计算信息用于后续的切分与局部锚点选择，从而

减少重复遍历整棵大树的开销，并保证多次运行时在相同输入下获得一致的局部分区定义。

全局骨架的构建

骨架构建阶段的目标是在整棵 ingroup 范围内选取一组能够代表全局拓扑骨干的样本，用于后续所有局部分析的共享参照。方法支持用户显式提供骨架样本集合，也支持在用户未完全指定时自动补齐。若用户提供的骨架样本数少于目标骨架规模，则这些指定样本会被完整保留，其余名额再由自动算法补充；若用户给定的样本数超过目标规模，则流程直接终止，以避免静默截断带来的不可控偏差。

自动骨架抽样采用一种确定性的前沿分区策略。具体而言，流程首先在 ingroup 中递归构建若干前沿 clade，使前沿簇的数量等于所需骨架规模；随后，从每个前沿簇内部选取一个代表性端点作为骨架样本。代表样本优先选择相对于该前沿祖先位置更 “深” 的终端，从而尽量覆盖多个大尺度系统发育区域，而不是在少数局部支系上过密采样。该设计的目标是提高不同局部子树之间的共享参考程度，降低目标子树被过度碎片化的风险。

目标子树的递归切分

在骨架确定后，主树的 ingroup 部分被递归切分为多个目标子树。切分准则不是总样本量，而是某一 clade 中非骨架样本的数量。只要某个 clade 所包含的非骨架样本数不超过预设容量上限，该 clade 即可直接作为一个目标子树；若超过上限，则继续向下递归直至满足约束。这里的容量上限由局部分析允许的最大样本量扣除全局骨架、全局外群以及局部锚点预留名额后得到，因此它实质上控制的是 “每个局部分析中真正需要重新估计的核心目标样本数”。

这一设计使得目标子树能够尽量保持原树中的单系结构，同时允许原始目标 clade 内部保留少量骨架后代。换言之，目标子树的原始结构定义与其真正负责的非骨架样本集合并不完全等同。流程会显式记录每个目标子树负责的非骨架样本集合，并检查所有非骨架样本是否被无重叠且无遗漏地分配给某一个目标子树，从而保证后续局部分析和回拼接具有完备覆盖。

局部重叠锚点与 PAML 子树构造

对每个目标子树，流程进一步构造用于局部最大似然分析的 PAML 子树。每棵 PAML 子树固定包含四类样本：全局骨架样本、该目标子树负责的非骨架样本、局部重叠锚点以及单一全局外群。其中，全局骨架和全局外群为所有局部分析共享，目标非骨架样本构成该子树的核心校正对象，而局部锚点则用于在相邻目标区域之间保留重叠信息，从而为后续的全局尺度统一提供额外约束。

局部锚点的选择遵循边界邻近原则。具体地，算法从目标子树根部向上回溯，在每一层父节点的兄弟支系中收集候选终端，并按照边界层级和局部系统发育距离进行排序。与目标边界更接近、且不属于骨架、外群或目标核心样本的候选端点优先被纳入锚点集合。该策略使局部锚点尽量来源于目标边界附近的相邻谱系，从而在不显著扩大子树规模的前提下，增加不同局部分析之间可比较的参考路径。

完成样本集合构建后，流程从标准化主树中诱导出对应的局部树，并再次以单一外群重定

根，从而生成满足最大似然分析要求的局部输入树。与此同时，系统还会生成清单表，逐一记录每个局部子树中的骨架样本、目标样本、锚点和外群，以保证每一棵局部树的样本组成可以被完全追溯。

切分阶段的结构验证

在切分完成后，流程会对骨架树、目标子树和 PAML 子树执行系统化验证。验证内容包括样本清单与摘要表是否一致、骨架树是否恰好包含骨架样本与外群、每棵 PAML 子树的端点集合与登记信息是否完全匹配、每棵目标子树是否完整覆盖其应负责的非骨架样本、以及所有非骨架样本在各目标子树之间是否实现唯一分配。对于原始目标子树，验证允许其含有额外的骨架后代，因为这些骨架后代属于保留原始系统发育上下文的一部分，而不属于目标样本定义错误。

除快速结构校验外，方法还保留严格复算式验证模式，即从原始输入重新执行一次分区逻辑，并将所得结果与磁盘上的现有产物逐项比对。这样的双层验证设计使常规运行保持较高效率，同时为开发和调试阶段提供更强的可再现性保障。

局部最大似然输入准备

局部最大似然分析的输入准备包括三部分：局部树文件、局部 FASTA 文件和控制参数文件。局部树文件被改写为目标软件所要求的标准格式，即首行给出样本数，第二行为单行 Newick 表达式。局部 FASTA 文件则从全局比对中提取与该局部子树端点完全对应的序列。为适应实际数据中可能存在的命名差异，样本名称既支持完全匹配，也支持基于前缀的回退匹配策略，从而在标签附带额外后缀的情况下仍能完成树与序列的对应。

控制参数文件采用模板渲染方式生成。方法只重写与当前作业直接相关的序列文件、树文件和输出文件路径，而不在程序内部硬编码模型、时钟、kappa、alpha 或数据清洗等参数。这样既保证了每个局部作业的输入输出路径完全确定，又保留了用户对最大似然模型设置的充分控制。

并行运行、断点续跑与局部结果解析

局部最大似然作业以并行方式调度，支持用户指定作业并发数以及单作业线程数。流程使用基于日志的断点续跑机制来记录成功和失败的作业，而不是通过扫描输出文件夹来推断运行状态。每个已完成作业都会写入成功日志，失败作业则写入失败日志，并通过文件锁避免并发写入竞争。该机制能够在中断、重跑或大批量任务环境下保持稳定、可恢复和可审计的运行状态。

在作业成功结束后，流程会解析最大似然输出文件，提取具名树、索引树、分支参数和终端枝长信息，并将具名树转化为后续合树所需的分析树。解析完成后，每棵分析树都必须再次通过端点集合与摘要信息的一致性校验，否则流程会将其标记为解析失败。通过这一过程，原始局部估计结果被统一转换为可合并、可比较且带有详细参数记录的标准化局部分析树。

可选的骨架专用估计

除常规目标子树分析外，方法还支持对骨架树单独执行一次最大似然估计。该步骤使用骨架树自身作为唯一分析对象，保留骨架样本与外群，并生成一个骨架专用分析结果。随后，骨架专用结果会被进一步规范为超度量树，以作为后续标准化阶段的可选全局参照。该设计使研究者能够在局部目标树之外额外获得一份仅依赖骨架的全局枝长尺度信息。

合树策略：骨架聚合与目标 graft

合树阶段以标准化主树拓扑为唯一全局参照，不允许局部结果修改全树拓扑。首先，方法根据目标子树定义，在主树中用占位符替代各目标区域，从而生成仅保留骨架、外群和目标占位节点的 scaffold。接着，对每一棵局部分析树执行两项操作。第一，利用局部树中与骨架相关的共享路径估计该局部树相对于主树的尺度因子。该尺度因子通过共享参考样本之间的成对距离比值计算得到，本质上反映了局部分析树与全局主树在路径长度尺度上的偏移程度。第二，从局部分析树中提取仅包含目标非骨架样本的严格单系目标 clade，作为后续 graft 的真实替换对象。

在完成局部尺度校正后，所有与骨架相关的分支会按照其“所张成的骨架后代集合”构造唯一签名。不同局部树中具有相同骨架签名的分支被视为对同一全局骨架边的重复观测，并通过加权几何平均进行聚合。权重由该局部树所承载的目标样本数决定，从而使信息量更大的局部分析对全局骨架枝长具有更高贡献。随后，每个目标 clade 会被 graft 回 scaffold 中对应的占位位置，最终生成一棵保留原始全树端点集合的合并树。该合并树仍为非超度量树，其主要用途是整合经过局部最大似然估计后的枝长信息。

合树阶段的质量控制

合树结束后，流程会验证 scaffold 是否恰好由骨架、外群和占位节点构成，验证最终合并树是否与主树具有完全相同的端点集合，并检查所有占位节点是否已经被真实目标 clade 替换。同时，系统还会检查每个目标子树是否恰好 graft 一次、骨架边长聚合结果是否完整、尺度因子报告是否覆盖所有目标分区，以及所有非根分支是否具有非负长度。只有在这些条件全部满足时，合并结果才被视为结构上有效。

超度量标准化

由于合并后的全局树不必天然满足超度量约束，流程最后增加一个超度量标准化阶段，在保持拓扑不变的前提下对枝长进行统一投影。方法支持两种模式。第一种模式仅延长终端枝，使所有端点到根的距离达到当前最大深度；此时内部枝长保持不变，因此调整最为保守。第二种模式基于受限节点高度拟合，先根据后序遍历估计每个内部节点的高度，再通过父子高度差重新计算所有分支长度。后者不仅调整终端枝，也可在最小正枝长约束下重新分配内部枝长，以得到更平滑的超度量结构。

无论使用哪一种模式，标准化阶段都不会改变树的根部结构、端点集合或内部拓扑关系，仅改变分支长度。系统会保存投影前后的根到端点距离报告，并记录每条边在投影前后的长度

变化，以便研究者区分哪些边被修改、修改幅度多大以及标准化后的树是否真正满足超度量要求。

最终验证与拓扑一致性比较

超度量树生成后，流程会进一步验证投影输入树和最终超度量树均保持统一外群定根结构，检查最终树是否在设定容差内满足超度量性质，并确认其端点集合与主树完全一致。同时，方法还对最终超度量树与标准化主树进行非平凡 clade 签名比较，以评估标准化过程是否保持了原始根树拓扑。通过这一拓扑比较，流程能够将 “枝长投影” 与 “拓扑改变” 严格区分开来，从而保证最终输出仍然忠实于原始分治框架所定义的全局系统发育结构。

开发辅助模式与可重复性保障

为支持在真实最大似然估计尚未全部完成时开发合树与标准化模块，流程还提供模拟与伪分析模式。前者通过对局部树枝长施加对数正态扰动来生成拓扑不变的模拟分析树，后者则直接构造可用于后续阶段的伪局部结果和对应状态表。这些模式使流程的不同阶段能够被解耦测试，从而加快方法开发并增强错误定位能力。

整体而言，本方法通过显式的中间文件、清单表、验证报告和结构化异常处理机制，将大树分治、局部最大似然估计、全局尺度统一和超度量投影整合为一套可重复、可追踪、可分阶段审计的系统发育枝长校正框架。

English

Overall framework

We implemented a divide-and-conquer workflow for branch-length correction on large phylogenetic trees. The central idea was to decompose the global problem into four consecutive stages: global backbone definition, local target-subtree inference, global regrafting, and final ultrametric standardization. Throughout the procedure, the topology of a single master tree was treated as the invariant reference, whereas branch lengths were re-estimated locally and subsequently reconciled on a shared global scale. Thus, the method was designed to preserve topology as a hard constraint while using local maximum-likelihood estimation primarily for branch-length refinement.

The workflow required a large phylogenetic tree, a multiple-sequence alignment matched to the terminal taxa, an explicit outgroup definition, and a model-control template for maximum-likelihood inference. The outputs included a normalized master tree, a backbone tree, target subtrees, PAML-ready local subtrees, local branch-length estimates, a globally merged non-ultrametric tree, and a final ultrametric tree. To maximize reproducibility, every stage produced explicit intermediate artifacts, manifests, and validation reports, thereby enabling each structural definition, sample assignment, and branch-length update to be independently audited.

Rooted master-tree preparation

The workflow began by determining whether the input tree was already rooted. If the tree was unrooted, strict outgroup rooting was performed using the user-defined outgroup set; if the tree was already rooted, the original root placement was retained. The resulting intermediate master tree was then structurally normalized so that the root satisfied a singleton-outgroup constraint and the topology was converted, whenever necessary, into a bifurcating form. Unary chains were collapsed, and multifurcations were resolved into binary structure by inserting zero-length intermediate nodes. This step was not intended to alter phylogenetic relationships, but rather to provide a uniform computational substrate for downstream subtree splitting, tree merging, and topology validation.

After rooting and bifurcation, stable internal node identifiers were assigned and root-distance metadata, tip order, and input fingerprints were recorded. These precomputed attributes were subsequently used during recursive partitioning and local anchor selection, reducing repeated traversals of the large master tree and ensuring deterministic behavior across repeated runs with the same inputs.

Construction of the global backbone

The purpose of the backbone stage was to define a set of globally representative taxa that would serve as shared anchors in all local analyses. The method supported both user-specified backbone taxa and automatic supplementation when the user-defined set was incomplete. If the provided backbone set contained fewer taxa than the requested backbone size, all user-defined taxa were retained and the remaining slots were filled automatically. If the provided set exceeded the requested backbone size, the workflow terminated explicitly rather than truncating the set silently, thereby avoiding uncontrolled bias.

Automatic backbone sampling followed a deterministic frontier-partition strategy. Briefly, the ingroup subtree was recursively decomposed into a number of frontier clades equal to the desired backbone size, after which a representative terminal was chosen from each frontier clade. The selected representative favored deeper descendants relative to the local frontier ancestor, thereby encouraging broad phylogenetic coverage across major regions of the tree instead of oversampling a small number of local lineages. This design was intended to increase the amount of globally shared reference structure and reduce excessive fragmentation of downstream target partitions.

Recursive partitioning of target subtrees

Once the backbone had been fixed, the ingroup component of the master tree was recursively partitioned into target subtrees. Partitioning was driven by the number of non-backbone descendants within a clade rather than by total subtree size. Any clade whose non-backbone descendant count did not exceed a predefined capacity was accepted directly as a target subtree; otherwise, recursion continued into its children until all non-backbone taxa had been assigned. This capacity corresponded to the maximum local subtree size after reserving slots for the global backbone, the singleton outgroup,

and a predefined number of local overlap anchors. In practice, it therefore regulated the number of taxa whose branch lengths were expected to be re-estimated locally.

This design allowed target subtrees to remain as close as possible to monophyletic clades in the original master tree while tolerating the presence of backbone descendants inside the raw target clade. Consequently, the raw target-clade definition was not identical to the set of non-backbone taxa for which that partition was ultimately responsible. The workflow explicitly recorded the non-backbone target set assigned to each partition and enforced complete, non-overlapping coverage of all non-backbone taxa, thereby ensuring that the collection of target subtrees formed an exhaustive partition of the non-backbone portion of the tree.

Local overlap anchors and construction of PAML-ready subtrees

For each target partition, a local subtree was constructed for maximum-likelihood analysis. Each local subtree contained four classes of taxa: the complete global backbone, the non-backbone taxa assigned to the focal target partition, a set of local overlap anchors, and the singleton global outgroup. The backbone and outgroup were shared across all local analyses, the target taxa represented the focal correction region, and the overlap anchors were introduced to preserve local comparability among adjacent target regions.

Local overlap anchors were chosen according to a boundary-proximity principle. Starting at the root of the target clade and moving upward toward the global root, the algorithm collected terminal candidates from sibling lineages encountered along the way. Candidates belonging to the backbone, the outgroup, or the focal target itself were excluded. The remaining candidates were ranked by boundary level and then by local phylogenetic distance to the target boundary, so that taxa closer to the target interface were preferentially retained as anchors. This strategy was designed to maximize informative local overlap while minimizing unnecessary expansion of local subtree size.

After the taxon set had been finalized, the corresponding local tree was induced directly from the normalized master tree and rerooted on the singleton outgroup. At the same time, the workflow generated explicit manifests describing the backbone taxa, target taxa, anchor taxa, and outgroup present in each local subtree, thereby making every local tree fully traceable at the sample-composition level.

Structural validation of the split stage

Following subtree construction, the workflow performed systematic validation of the backbone tree, target subtrees, and PAML-ready local subtrees. Validation included consistency checks between manifests and summary tables, confirmation that the backbone tree contained exactly the backbone taxa plus the outgroup, verification that every local analysis subtree matched its registered taxon set, and confirmation that each raw target subtree fully represented its assigned non-backbone taxa. Because raw target subtrees intentionally preserved their original phylogenetic context, the presence of additional backbone descendants in those raw target clades was treated as valid rather than erroneous.

In addition to rapid structural checks, the method retained a strict recomputation mode in which the

partitioning logic was rerun from the original inputs and all derived products were compared against the materialized outputs on disk. This two-tier validation scheme allowed routine runs to remain efficient while preserving a stronger reproducibility mode for development and debugging.

Preparation of local maximum-likelihood inputs

Preparation of local maximum-likelihood jobs consisted of three elements: local tree files, local FASTA files, and control files. Each local tree was rewritten into the target software's expected format, with the first line indicating the number of taxa and the second line containing a single-line Newick representation. Each local FASTA file was then generated by extracting exactly those sequences corresponding to the taxa present in the local subtree. To accommodate realistic labeling discrepancies, taxon-to-sequence mapping supported both exact matching and a prefix-based fallback strategy.

Control files were generated by template rendering. Only the fields directly tied to the current job, namely the sequence file, tree file, and output file, were rewritten automatically. Model parameters such as substitution model, clock setting, kappa, alpha, and data-cleaning behavior were intentionally left under user control via the supplied template. This design ensured deterministic per-job I/O while preserving full flexibility in model specification.

Parallel execution, resume control, and parsing of local results

Local maximum-likelihood jobs were dispatched in parallel, with user-configurable job concurrency and per-job thread allocation. The workflow used a log-based resume mechanism rather than inferring job status by scanning output directories. Successful jobs were recorded in a success log, failed jobs in a failure log, and file locking was used to prevent race conditions under concurrent execution. This design provided a stable and auditable mechanism for interruption recovery and large-scale batch execution.

After successful completion, each local result was parsed to extract the named tree, indexed tree, branch-level parameters, and terminal branch lengths. The named tree was converted into a standardized analysis tree for downstream merging, and every parsed tree was validated against the expected taxon set and recorded summary information. Any discrepancy in taxon composition or taxon count caused the parse stage to fail explicitly. Through this step, raw local outputs were transformed into merge-ready analysis trees accompanied by detailed branch-level metadata.

Optional backbone-only inference

In addition to the standard target-subtree analyses, the workflow supported an optional backbone-only maximum-likelihood analysis. In this mode, the backbone tree itself was used as a single standalone inference target while retaining both the backbone taxa and the outgroup. The resulting backbone-specific estimate was then normalized into an ultrametric tree and could be used as

an optional global branch-length reference during later stages. This feature allowed the method to derive a branch-length scale from the backbone alone, independently of the target-partition results.

Global merge through backbone aggregation and target grafting

During the merge stage, the normalized master-tree topology remained the sole global topological reference and could not be altered by local results. First, a scaffold was constructed by replacing each target region in the master tree with a placeholder while retaining the backbone, the outgroup structure, and any non-target descendants that had to remain attached at the target root. Each local analysis tree then contributed in two distinct ways. First, a scale factor was estimated from path-length ratios between shared reference taxa in the local analysis tree and the master tree. These shared references consisted of global backbone taxa, local overlap anchors, and the outgroup. The resulting scale factor captured the relative path-length offset between the local estimate and the global reference.

Second, the exact monophyletic clade containing the target partition's non-backbone taxa was extracted from the local analysis tree and used as the actual graft object. For backbone-associated edges, a unique signature was defined by the set of descendant backbone taxa subtended by a branch. Branches from different local trees sharing the same backbone signature were treated as repeated observations of the same global backbone edge and were aggregated by a weighted geometric mean, with weights determined by the number of target taxa contributed by each local subtree. After this global backbone aggregation, each extracted target clade was grafted back onto the scaffold at its designated placeholder, producing a merged global tree that preserved the original taxon set and topology while integrating branch-length information estimated locally.

Quality control of the merge stage

Following the merge, the workflow verified that the scaffold contained exactly the backbone taxa, the outgroup, and placeholder nodes; that the final merged tree contained exactly the same terminal taxa as the master tree; and that all placeholders had been replaced by genuine target clades. The workflow also verified that each target partition had been grafted exactly once, that backbone edge aggregation was complete, that subtree scale-factor reports covered all target partitions, and that all non-root branches had non-negative lengths. Only when all of these conditions were satisfied was the merged result considered structurally valid.

Ultrametric standardization

Because the merged global tree was not required to be ultrametric, a final standardization stage projected branch lengths onto an ultrametric tree while preserving topology. Two projection modes were implemented. In the first mode, only terminal branches were extended so that all tips reached the maximum observed root-to-tip depth, while internal branches remained unchanged. This represented the most conservative adjustment. In the second mode, node heights were estimated under

a constrained-height formulation, after which branch lengths were recalculated from parent-child height differences under a minimum positive branch-length constraint. This second mode allowed both terminal and internal branches to be redistributed more smoothly.

In both cases, the procedure preserved the rooted structure, the tip set, and the internal topology, altering branch lengths only. The workflow recorded root-to-tip depth summaries before and after projection and stored edge-wise branch-length changes so that investigators could identify which branches had been modified, by how much, and whether the final tree satisfied the desired ultrametric criterion.

Final validation and topology comparison

After ultrametric projection, the workflow verified that both the projection input tree and the final ultrametric tree retained the singleton-outgroup root structure, that the final tree satisfied ultrametricity within a predefined tolerance, and that its tip set matched that of the master tree exactly. In addition, the final ultrametric tree was compared to the normalized master tree using non-trivial clade signatures in order to assess whether the projection step had preserved rooted topology. This comparison ensured that branch-length projection and topology modification were rigorously distinguished, so that the final output remained faithful to the original divide-and-conquer phylogenetic structure.

Development support and reproducibility safeguards

To facilitate method development before all real maximum-likelihood analyses were available, the workflow also supported simulation and pseudo-analysis modes. In one mode, topology-preserving surrogate analysis trees were generated by applying lognormal perturbations to local branch lengths. In another mode, pseudo-local outputs and synchronized status tables were generated directly for downstream development. These auxiliary modes allowed the split, merge, and standardization components to be tested independently, thereby accelerating development and improving fault isolation.

Taken together, the method integrates large-tree decomposition, local maximum-likelihood branch-length estimation, global scale reconciliation, and final ultrametric projection into a reproducible and auditable framework. Reproducibility was enforced not through hidden in-memory state, but through explicit intermediate artifacts, structured manifests, staged validation reports, and deterministic definitions of each local and global operation.

Methods

We developed a divide and conquer phylogenetic workflow to estimate branch lengths and perform time calibration for a large tree that was not practical to analyze directly with PAML. The input Newick tree was first rooted using a predefined singleton outgroup and normalized to a rooted binary topology. We then selected 100 representative ingroup tips as a global backbone using a greedy farthest patristic sampling strategy to maximize phylogenetic coverage across the tree. All remaining non backbone tips were recursively partitioned into monophyletic target clades. Each local subtree

contained at most 300 tips, including the full backbone, one outgroup tip, the target clade, and up to 16 local overlap anchors. Local anchors were selected from regions adjacent to each target clade using a nearest boundary patristic strategy, thereby preserving shared taxa across neighboring subtrees for later scale harmonization.

For each local subtree, we generated an independent baseml input set, including a subtree specific tree file, a FASTA file extracted from the full alignment, and a control file rendered from a shared template. Branch lengths were estimated with baseml under the REV nucleotide substitution model with a global molecular clock and four discrete gamma categories for among site rate heterogeneity, with kappa and alpha estimated empirically. In addition to the local analyses, a separate baseml run was performed on the tree containing only the backbone and the outgroup. The resulting backbone tree was then normalized to an ultrametric form and used as the fixed global reference for downstream merging.

Full tree reconstruction was achieved through a backbone constrained grafting strategy. The original rooted master tree was first converted into a scaffold tree, in which each target clade was replaced by a placeholder branch while the backbone structure was preserved. Each local tree was then rescaled to the global backbone reference using pairwise patristic distances among shared backbone taxa. The corresponding target clade was further adjusted under a redline depth constraint so that its maximum depth matched the available grafting depth at the placeholder position, and was then grafted back into the scaffold. Repeating this procedure for all target clades yielded a merged maximum likelihood tree. This merged tree was subsequently projected to a globally ultrametric form using a constrained height fit procedure, and converted from substitution units to years by node calibration using the RSRS lineage, with the root to RSRS divergence fixed at 180000 years. Quality control was applied throughout the workflow to verify topological consistency, completeness of tip recovery, validity of branch lengths, and compliance with ultrametric constraints.